

Software Testing

ACRONYMS

AI	Artificial Intelligence
API	Application Program Interface
ARINC	Aeronautical Radio Incorporated
ATDD	Acceptance Test-Driven Development
CMMI	Capability Maturity Model Integration
CSS	Cascading Style Sheets
DICOM	Digital Imaging and Communications in Medicine
DL	Deep Learning
DU	Definition and Use
EBSE	Evidence-Based Software Engineering
ETSI	European Telecommunications Standards Institute
FHIR	Fast Healthcare Interoperability Resources
GDPR	General Data Protection Regulation
GPS	Global Positioning System
GUI	Graphical User Interface
HIL	Hardware-In-the-Loop
HIPAA	Health Insurance Portability and Accountability Act
HL7	Health Level Seven
IoT	Internet of Things
KPI	Key Performance Indicator
MC/DC	Modified Condition Decision Coverage
ML	Machine Learning
MTTR	Mean Time to Recovery

OAT	Orthogonal Array Testing
ODC	Orthogonal Defect Classification
SoS	System of Systems
SPI	Software Process Improvement
SPICE	Software Process Improvement and Capability Determination
SUT	System Under Test
TDD	Test-Driven Development
TMMi	Test Maturity Model integration
UI	User Interface
UP	Unified Process

INTRODUCTION

Software testing consists of the *dynamic* validation that a *system under test (SUT)* provides *expected* behaviors on a *finite* set of *test cases* suitably *selected* from the usually infinite execution domain.

In the above statement, italicized words correspond to key issues in the Software Testing knowledge area (KA). Those terms are discussed below.

- *System Under Test*: This term refers to the tested object, which can be a program, a software product, an application, a service-oriented application (e.g., web services, microservices), middleware (HW/SW), a services composition, a system, a System of Systems (SoS), or an Ecosystem.
- *Test Case*: A test case is the specification of all the entities that are essential for the execution, such as input values, execution and timing conditions, testing procedure,

and the expected outcomes (e.g., produced values, state changes, output messages). Input values alone are not always sufficient to specify the test cases because the SUT might react to the same input with different behaviors, depending, for instance, on the SUT state or environmental conditions. A set of test cases is usually called a *test suite*.

- *Dynamic*: Dynamic validation requires executing a test suite on the SUT. Static techniques complement dynamic testing, and they are covered in the Software Quality KA.¹
- *Finite*: Even in a simple SUT, executing all the possible test cases (i.e., exhaustive testing) could require months or years. Consequently, in practice, testing targets a subset of all possible test cases determined by different criteria. Testing always implies a trade-off between limited resources and schedules on the one hand and inherently unlimited test requirements on the other.
- *Selected*: Identifying the most suitable selection criteria under given conditions is a complex problem. Different techniques can be considered and combined to tackle that problem, such as risk analysis, software requirements, cost reduction, quality attributes satisfaction, prioritization, and fault detection. The many proposed test techniques differ in how the test suite is selected, and software engineers must be aware that different selection criteria might yield vastly different degrees of effectiveness.
- *Expected*: For each executed test case, it must be possible, although it might not be easy, to decide whether the observed SUT outcomes match the expected ones. Indeed, the observed behavior may be checked against user needs (commonly referred to as testing for validation), against a specification (testing for verification), or, perhaps, against the foreseen behavior from

implicit requirements or expectations. (See Section 4.3, Acceptance Criteria-Based Requirements Specification, in the Software Requirements KA.)

As reflected in this discussion, software testing is a pervasive and holistic activity involving all the steps of any process development life cycle (e.g., traditional or shift-left development). The remainder of this chapter presents the basics of software testing and its challenges, issues, and commonly accepted practices and solutions.

BREAKDOWN OF TOPICS FOR SOFTWARE TESTING

Figure 5.1 shows the breakdown of topics for the Software Testing KA. The Matrix of Topics vs. Reference Material provides a more detailed breakdown at the end of this KA. The first topic, Software Testing Fundamentals, covers the basic definitions in software testing, the basic terminology and key issues, and software testing's relationship with other activities.

The second topic, Test Levels, contains two (orthogonal) subtopics. The first subtopic, The Target of the Test, lists the levels into which the testing of large software is traditionally subdivided, and the second subtopic, Objectives of Testing, discusses testing for specific conditions or properties. Not all types of testing apply to every software product, nor has every possible type been listed. The Target of the Test and Objectives of Testing together determine how the test suite is identified, both regarding its consistency (How much testing is enough for achieving the stated objective?) and its composition (Which test cases should be selected for achieving the stated objective?). (However, usually, “for achieving the stated objective” remains implicit, and only the first part of the two questions above is posed.) Criteria for addressing the first question are

¹ It is worth noting that terminology is not uniform among different communities, and some use the term *testing* to refer to static techniques as well.

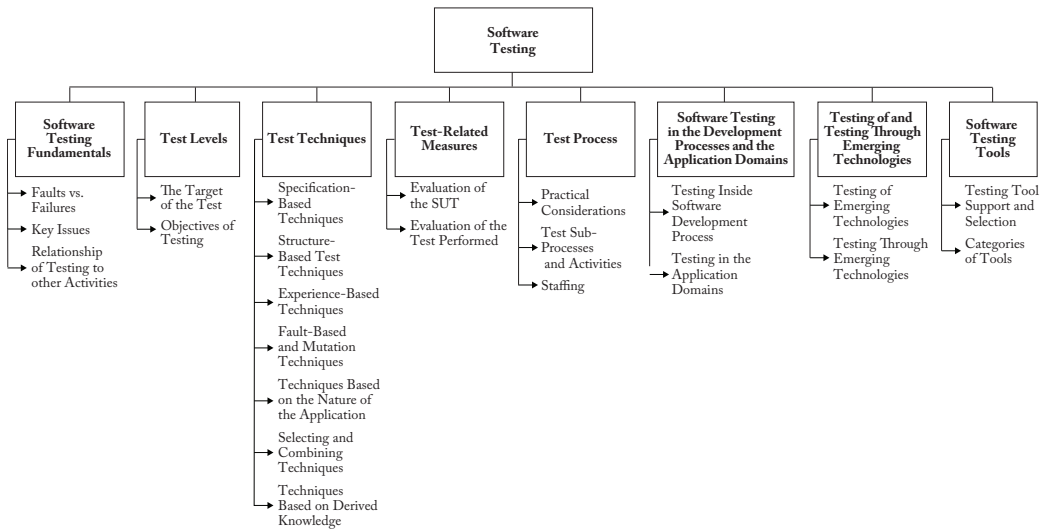


Figure 5.1. Breakdown of Topics for the Software Testing KA

test adequacy criteria, whereas those used for addressing the second question are the *test selection criteria*.

Several Test Techniques have been developed in the past few decades, and new ones are still being proposed. Therefore, the third topic covers generally accepted and standardized techniques.

Test-Related Measures are dealt with in the fourth topic, while the issues relative to the Test Process are covered in the fifth.

Software Testing in the Development Processes and the Application Domains is described in the sixth topic, and Testing of and Testing Through Emerging Technologies are described in the seventh topic. Finally, Software Testing Tools are presented in topic eight.

1. Software Testing Fundamentals

[1*, c1, c2; 2*, c8; 14*, c7]

This section provides an overview of the main testing issues and the relationship of testing to the other activities. Most of the testing terms used here are also defined. A more comprehensive overview of the testing and

testing-related terminology can be found in the cited references.

1.1 Faults vs. Failures

[1*, c1s5; 2*, c1; 14*, c1s3]

Many terms are used in the software engineering literature to describe a malfunction: notably *fault* (see, for comparison, *defect* in Section 3.2, Defect Characterization, in the Software Quality KA), *failure* and *error*. It is essential to distinguish between the *cause* of a malfunction (for which the term *fault* is used here) and an undesired effect observed in the system's delivered service (a *failure*). Indeed, there might well be faults in the software that never manifest as failures. (See Theoretical and Practical Limitations of Testing in Section 1.2.8.) Thus, testing can reveal failures, but the faults causing them are what can and must be removed. However, a failure's cause cannot always be unequivocally identified. No theoretical criteria exist to definitively determine, in general, the fault that caused an observed failure. The fault might have to be modified to remove the failure, but other modifications

might also work. To avoid ambiguity, we could refer to *failure-causing inputs* instead of faults — those sets of inputs that cause a failure to appear.

1.2. Key Issues

This subsection provides an overview of the main testing issues.

1.2.1. Test Case Creation

[1*, c12s1, c12s3, 2*, c8]

Test case creation or *generation* creates the test suite useful for testing the SUT for specific purposes (e.g., adequacy, accuracy, or assessment). Because test case generation is among the most important and intensive software testing activities, it is usually supported by approaches, techniques, and tools to automate the process.

1.2.2. Test Selection and Adequacy Criteria

[1*, c1s14, c6s6, c12s7, 2*, c8]

A test selection criterion is a means of selecting test cases or determining that a test suite is sufficient for a specified purpose. Test case selection aims to reduce the cardinality of the test suites while keeping the same effectiveness in terms of coverage or fault detection rate. Test adequacy criteria can be used to decide when sufficient testing is accomplished.

1.2.3 Prioritization/Minimization

[4, part 2, part 3, c5]

Suitable strategies for test case selection or prioritization can be adopted to improve testing efficacy. Test case *prioritization* aims to define a test execution order according to some criteria (e.g., coverage, fault detection rate, similarity, and risk), so those tests with a higher priority are executed before those with a lower priority. Test case *minimization* usually aims to reduce a test suite by removing redundant test cases according to some criterion or purpose.

1.2.4. Purpose of Testing

[1*, c13s11, c11s4, 2*, c8]

Different well-defined purposes can guide testing activity; it is only by considering a specific purpose that a test suite can be generated (selected), executed, and evaluated (see Section 2 for more details).

1.2.5. Assessment and Certification

[4, part 1, c5; 2*, c7, c25; 8]

Testing needs to focus on specific (mandatory) prescriptions, such as requirements, laws, and standards. Test cases should be generated and executed to provide evidence useful for evaluating and/or certifying adherence to the selected prescriptions. Usually, *assessment* and *certification* of the test results include verifying that the test cases have been derived and generated using baseline requirements, adopting a configuration control process, and using repeatable processes.

1.2.6. Testing for Quality Assurance/ Improvement

[1*, c16s2; 4, part 1, c5; 9]

Testing has many aspects, including quality improvement and assurance. These characteristics involve planned and systematic supporting processes and activities leveraging confidence that the SUT fulfills established technical or quality requirements. Thus, quality *improvement* and *assurance* involve defining methods, tools, skills, and practices to achieve the specific quality level and objectives. The list of the main quality characteristics that testing can measure or assess is reported in ISO/IEC 25010:2023 [9]. (See also Section 1.3.2, Software Product Quality, in the Software Quality KA.)

1.2.7. The Oracle Problem

[1*, c1s9, c9s7]

An important testing component is the oracle. Indeed, a test is meaningful only if it is possible to decide its observed outcome.

An *oracle* can be any human or mechanical agent that decides whether the SUT behaved correctly in each test and according to the expected outcomes. Consequently, the oracle provides a “pass” or “fail” verdict. The oracle cannot always decide; in these cases, the test output is classified as inconclusive. There are many kinds of oracles — for example, unambiguous requirements specifications, behavioral models, and code annotations. The automation of oracles can be difficult and expensive.

1.2.8. Theoretical and Practical Limitations

[1*, c2s7]

Testing theory warns against ascribing unjustified confidence to a series of successful tests. Unfortunately, most established results of the testing theory are negative results in that they state what is not achieved as opposed to what is achieved. The most famous quotation on this point is the Dijkstra aphorism that “program testing can be used to show the presence of bugs, but never to show their absence” [3]. The obvious reason for this is that complete testing is not feasible in realistic software.

1.2.9. The Problem of Infeasible Paths

[1*, c4s7]

Infeasible paths are control flow paths that cannot be exercised by any input data (i.e., test cases). Managing (*i.e.*, identifying, solving or removing) the infeasible paths can help reduce the time and resources devoted to testing. They are a significant problem in path-based testing, particularly in the automated derivation of test cases to exercise control flow paths. Additionally, the detection of infeasible paths can also play a role in reducing security vulnerabilities.

1.2.10. Testability

[1*, c17s2]

The term *software testability* has two related but different meanings. On the one hand, it

refers to the ease with which a given test coverage criterion can be satisfied; on the other hand, it is defined as the likelihood, possibly measured statistically, that a test suite will expose a failure *if* the software is faulty. Both meanings are important.

1.2.11 Test Execution and Automation

[4, part 1, c4]

An important challenge of testing is to improve attainable automation, either by developing advanced techniques for generating the test inputs or, beyond test generation, by finding innovative support procedures to (fully) automate the different testing activities — for instance, to increase the number of test cases generated or executed.

1.2.12. Scalability

[1*, c8s7]

Scalability is the software’s ability to increase and scale up on its nonfunctional requirements, such as load, number of transactions, and volume of data. Scalability is also connected to the complexity of the platform and environment in which the program runs, such as distributed, wireless networks and virtualized environments, large-scale clusters, and mobile clouds.

1.2.13 Test Effectiveness

[1* c1s1; 2* c8s1; 8]

Evaluating the SUT, measuring a testing technique’s efficacy, and judging whether testing can be stopped are important evidences for software testing, and they may require defining and selecting the proper test effectiveness measures.

1.2.14 Controllability, Replication, and Generalization

[1* c12s12; 4, part 2, c7]

Specific aspects of testing include the following:

- *Controllability* refers to the transition of testing activities from the laboratory (i.e., controlled conditions) to reality (i.e., uncontrolled conditions).
- *Replication* refers to the ability for different people to perform the same testing activities. The purpose is to verify whether a given testing theory works, at least in the laboratory.
- The *generalization* of testing is connected to external validity — i.e., the extent to which the test approach can be applied to broader settings or target populations. The generalizability of the software testing can be important for managing the testing activities (in terms of cost and effort) and increasing confidence in the test results.
- Testing vs. Correctness Proofs and Formal Verification: See the Software Engineering Models and Methods KA.
- Testing vs. Debugging: See Construction Testing in the Software Construction KA and Debugging Tools and Techniques in the Computing Foundations KA.
- Testing vs. Program Construction: See Construction Testing in the Software Construction KA.
- Testing vs. Security: See the new KA: Software Security.
- Testing vs. Effort Estimation: See the Software Engineering Management KA.
- Testing vs. Legal Issues: See the Software Engineering Professional Practice KA.

1.2.15. Offline vs. Online Testing

[10, c3]

The testing process can be executed in two settings: *offline* and *online*. Usually, the SUT is validated in an environment without external interaction in offline testing, whereas the SUT interacts with the real application environment in online testing. The test cases are either manually or automatically derived in both cases, and the expected outcomes are used to assess the SUT.

1.3. Relationship of Testing to Other Activities

Software testing is related to but different from static software quality management techniques, proofs of correctness, debugging, and program construction. However, it is informative to consider testing from the viewpoint of software quality analysts and certifiers. For further discussion, see the following:

- Testing vs. Static Software Quality Management Techniques: See Section 2.2.1, Static Analysis Techniques, in the Software Quality KA.
- Testing vs. Quality Improvement/Assurance: See Section 1.3.2, Software Product Quality, in the Software Quality KA.

2. Test Levels

[1*, c1s13; 2*, c8s1]

Software testing is usually performed at different *levels* throughout development and maintenance. Levels can be distinguished based on the object of testing, the *target*, or on the purpose or *objective* (of the test level).

2.1. The Target of the Test

[1*, c1s13, 2*, c8s1]

The target of the test can vary depending on the SUT, the conditions of the environment, and the budget/time devoted to the testing activity. Four test stages can be distinguished: unit, integration, system, and acceptance. These four test stages do not imply any development process, nor is any one of them assumed to be more important than the other three.

2.1.1. Unit Testing

[1*, c3, 2*, c8]

Unit testing verifies the functioning in isolation of SUT elements that are separately testable. Depending on the context, these could be the individual subprograms or components, a subsystem, or a composition of SUT components. Typically, but not always, the person who wrote the code conducts the unit testing.

2.1.2. Integration Testing

[1*, c7, 2*, c8]

Integration testing verifies the interactions among SUT elements (for instance, components, modules, or subsystems). Integration strategies involve the incremental (and systematic) integration of the SUT elements considering either identified functional threads or architecture specifications. Typical integration testing strategies are top-down, bottom-up, mixed (or sandwiched), and the big bang. They focus on different perspectives of the level at which SUT elements are integrated. Integration testing is a continuous activity that can be performed at each development stage. It may target different aspects, such as interoperability (e.g., compatibility or configuration) of the SUT elements or with the external environment. External interfaces to other applications, utilities, hardware devices or operating environments can also be considered.

2.1.3. System Testing

[1*, c8, 2*, c8]

System testing concerns testing the behavior of the SUT (according to the definition of Section 1). Effective unit and integration testing should have identified many SUT defects. In addition, system testing is usually considered appropriate for assessing non-functional system requirements, such as security, privacy, speed, accuracy, and reliability. (See Functional and Non-Functional Requirements in the Software Requirements KA and Software Quality Requirements in the Software Quality KA.)

2.1.4. Acceptance Testing

[1*, c1s7, 2*, c8s4]

Acceptance testing targets the deployment of a SUT. Its main goal is to verify that the SUT satisfies the requirements and the end-users' expectations. Generally, it is run by or with the end-users to perform those functions and

tasks for which the software was built. For example, this testing activity could target usability testing or operational acceptance. Defining acceptance tests before implementing the corresponding functionality is a key activity of the acceptance test-driven development (ATDD). (See the Software Requirements KA, Section 4.3.)

2.2. Objectives of Testing

[1*, c1s7]

Testing is conducted considering specific *objectives*, which are stated (more or less) explicitly and with varying degrees of precision. Stating the testing objectives in precise, quantitative terms supports measurement and control of the test process.

Testing can be aimed at verifying different properties. For example, test cases can be designed to check that the functional specifications are correctly implemented, which is variously referred to in the literature as conformance testing, correctness testing or functional testing. However, several other non-functional properties may be tested as well, including performance, reliability, and usability. (See Models and Quality Characteristics in the Software Quality KA.)

Other important testing objectives include but are not limited to reliability measurements, identification of security and privacy vulnerabilities, and usability evaluation; different approaches would be necessary depending on the objective. Note that, in general, the test objectives vary with the test target; different purposes are addressed at different levels of testing.

The subtopics listed below are those most cited in the literature.

2.2.1. Conformance Testing

[1*, c10s4]

Conformance testing aims to verify that the SUT conforms to standards, rules, specifications, requirements, design, processes, or practices.

2.2.2 Compliance Testing

[1*, c12s3]

Compliance testing aims to demonstrate the SUT's adherence to a law or regulation. Usually, compliance testing is forced by an external regulatory body.

2.2.3. Installation Testing

[1*, c12s2]

Often, after system and acceptance testing is completed, and the SUT has been installed in the target environment, the SUT is verified. *Installation testing* can be viewed as system testing conducted in the operational environment of hardware configurations and other operational constraints. Installation procedures may also be verified.

2.2.4. Alpha and Beta Testing

[1*, c13s7, c16s6, 2*, c8s4]

Before the SUT is released, it is sometimes given to a small, selected group of potential users for trial use (*alpha testing*) and/or to a larger set of representative users (*beta testing*). These users report problems with the product. Alpha testing and beta testing are often uncontrolled and are not always referred to in a test plan.

2.2.5. Regression Testing

[1*, c8s11, c13s3; 4, part 1, c5]

According to the definition reported in [5], *regression testing* is the “selective retesting of a SUT to verify that modifications have not caused unintended effects and that the SUT still complies with its specified requirements.” In practice, the approach is designed to show that the SUT still passes previously passed tests in a test suite (in fact, it is sometimes referred to as *non-regression testing*). In some cases, a trade-off must be made between the assurance given by regression testing every time a change is made and the resources required to perform the regression tests. This can be quite time-consuming because of the many tests

that might be executed. Regression testing can be conducted at each test level described in Section 2.1. It may involve functional and non-functional testing, such as reliability, accessibility, usability, maintainability, conversion, migration, and compatibility testing.

Regression testing may involve selection (see Section 1.2.2) and minimization (see Section 1.2.3) of test cases, as well as the adoption of prioritization approaches (see Section 2.2.6) to existing test suites.

Regression testing is a fundamental activity of Agile, DevOps, test-driven development (TDD), and Continuous Development. It is usually performed after integration testing and before deployment to production or operation.

2.2.6. Prioritization Testing

[1*, c12s7]

Test case prioritization aims to schedule test cases to increase the rate and likelihood of fault detection, the coverage of code under test, and the SUT's reliability. Typically, prioritization testing relies on heuristics, and its performance might vary according to the SUT, the environment, and the available test cases. Among the different prioritization proposals, similarity-based prioritization is one of the most commonly adopted. In this approach to prioritization, test cases are prioritized starting from those most dissimilar according to a predefined distance function.

2.2.7. Non-functional Testing

[2*, c8]

Non-functional testing targets the validation of non-functional aspects (such as performance, usability, or reliability), and it is performed at all test levels. At the state of the practice, there are hundreds of non-functional testing techniques that include but are not limited to the following:

- Performance Testing [4, part 1]: Performance testing verifies that the software meets the specified performance requirements and assesses performance

characteristics (e.g., capacity and response time).

- Load Testing [4, part 1]: Load testing focuses on validating the SUT's behavior under load pressure conditions to discover problems (e.g., deadlocks, racing, buffer overflows and memory leaks) or reliability, stability, or robustness violations. It aims to assess the rate at which different service requests are submitted to the SUT.
- Stress Testing [1*, c8s8]: Stress testing aims to push the SUT beyond its capabilities by generating a load greater than what the system is expected to handle.
- Volume Testing [4, part 1]: Volume testing targets the assessment of the SUT's internal storage limitations and its ability to exchange data and information.
- Failover Testing [1*, c17s2; 2*, c8]: Failover testing validates the SUT's ability to manage heavy loads or unexpected failure to continue typical operations (e.g., by allocating extra resources). Failover testing is also connected with recoverability validation.
- Reliability Testing [1*, c15; 2*, c11]: Reliability testing evaluates the SUT's reliability by identifying and correcting faults. Reliability testing observes the SUT in operation or exercises the SUT by using test cases according to statistical models (operational profiles) of the different users' behaviors. Usually, reliability is assessed through reliability growth models. The continuous development processes (such as DevOps) are facilitating the adoption of reliability testing in the various iterations for improving final SUT quality.
- Compatibility Testing [4, part 1; 10, c3]: Compatibility testing is used to verify whether the software can collaborate with different hardware and software facilities or with different versions or releases.
- Scalability Testing [1*, c8s7; 2* c17]: Scalability testing assesses the software's ability to scale up non-functional requirements such as load, number of

transactions, volume of data. It could integrate or extend load, elasticity and stress testing.

- Elasticity Testing [17]: Elasticity testing assesses the ability of the SUT (such as cloud and distributed systems) to rapidly expand or shrink compute, memory, and storage resources without compromising the capacity to meet peak utilization. Some elasticity testing objectives are to control behaviors, to identify the resources to be (un)allocated, and to coordinate events in parallel.
- Infrastructure Testing [8, annex H]: Infrastructure testing tests and validates infrastructure components to reduce the chances of downtime and improve the performance of the IT infrastructure.
- Back-to-Back Testing [5]: ISO/IEC/IEEE 24765 defines back-to-back testing as "testing in which two or more variants of a program are executed with the same inputs, the outputs are compared, and errors are analyzed in case of discrepancies."
- Recovery Testing [1*, c14s2]: Recovery testing is aimed at verifying software restart capabilities after a system crash or other disaster.

2.2.8. Security Testing

[2*, c13; 4, part 4, annex A]

Security testing focuses on validating that the SUT is protected from external attacks. More precisely, it verifies the confidentiality, integrity, and availability of the systems and their data. Usually, security testing includes validation against misuse and abuse of the software or system (negative testing). (See Security Testing in the Software Security KA.)

2.2.9. Privacy Testing

[2*, c13, c14]

Privacy testing is devoted to assessing the security and privacy of users' personal data to prevent attacks. It specifically assesses privacy

and information-sharing policies, as well as the validation of decentralized management of users' social profiles and data storage solutions. (See Legal Issue in the Software Engineering Professional Practice KA.)

2.2.10. Interface and Application Program

Interface (API) Testing

[2*, c8s1; 14*, c7s12; 4, part 5, c4, c7]

Interface defects are common in complex systems. *Interface testing* aims to verify whether the components' interface provide the correct exchange of data and control information. Usually, the test cases are generated from the interface specification. A specific interface testing objective is to simulate the use of APIs by end-user applications. That involves generating parameters of the API calls, setting conditions of the external environment, and defining internal data that affect the API.

2.2.11. Configuration Testing

[1*, c8s5]

Where the SUT is built to serve different users, *configuration testing* verifies the software under specified configurations.

2.2.12. Usability and Human-Computer

Interaction Testing

[2* c8s4; 19*, c6; 4, part 4, annex A]

The main task of *usability and human-computer interaction testing* is to evaluate how easy it is for end-users to learn to use the software. It might involve testing the software functions that support user tasks, the documentation that aids users, and the system's ability to recover from user errors. (See User-Centered Design in the Software Design KA.)

3. Test Techniques

[1*, c1s15; 4, part 4]

To increase the SUT's overall quality [4, part 4] available techniques propose systematic procedures and approaches.

Testing techniques can be classified by considering different key aspects such as specification, structure, and experience [4, part 4]. Additional classification sources can be the faults to be discovered, the predicted use, the models, the nature of the application, or the derived knowledge. For instance, model-based testing [7; 4, part 1] refers to all the testing techniques that use the concept of a model representing behavioral specification, the SUT's structure, or the available knowledge and experience. However, classification overlapping is possible, and one category might deal with combining two or more techniques.

Alternative classifications based on the degree of information about the SUT are available in the literature. Indeed, in the specification-based techniques, also known as *black-box* techniques, the generation of test cases is based only on the SUT's input/output behavior, whereas in the structure-based, also called *white-box* (or *glass-box* or *clear-box*), techniques, the test cases are generated using the information about how the SUT has been designed or coded.

As some testing techniques are used more than others, the remainder of the section presents the standard testing techniques and those commonly adopted at the state of the practice.

3.1. Specification-Based Techniques

[1*, c6s2; 4, part 4]

The underlying idea of *specification-based techniques* (sometimes also called domain testing techniques) is to select a few test cases from the input domain that can detect specific categories of faults (also called domain errors). These techniques check whether the SUT can manage inputs within a certain range and return the required output.

3.1.1. Equivalence Partitioning

[1*, c9s4]

Equivalence partitioning involves partitioning the input domain into a collection of subsets (or equivalence classes) based on a specified

criterion or relation. This criterion or relation can rely on the computational results, the control flow or data flow, or the valid inputs that are accepted and processed by the SUT and invalid inputs. An example can be the valid and out-of-range values. This last could generate an error message or initiate error processing. A representative test suite (sometimes containing only one test case) is usually taken from each equivalence class.

3.1.2. Boundary Value Analysis

[1*, c9s5; 4, part 4]

Test cases are chosen on or near the boundaries of the input domain of variables, with the underlying rationale that many faults tend to concentrate near the extreme values of inputs. An extension of this technique is *robustness testing*, wherein test cases are also chosen outside the input domain of variables to test program robustness in processing unexpected or erroneous inputs.

3.1.3. Syntax Testing

[1*, c10s11, 2*, c5; 4, part 4]

The Syntax Testing techniques, also known as formal specification-based techniques, rely on the SUT specifications in a formal language. (See Formal Methods in the Software Engineering Models and Methods KA.) This representation permits automatic derivation of functional test cases and, at the same time, provides an oracle for checking test results.

3.1.4. Combinatorial Test Techniques

[1*, c9s3; 4, part 4]

The Combinatorial Test Techniques systematically derive the test cases that cover specific parameters of values or conditions. According to [4, part 4], the commonly used combinatorial test techniques are All combinations Testing, Pair-Wise Testing, Each Choice Testing, and Base Choice Testing. All combinations testing focuses on all the possible input combinations, whereas its subset, also called *t-wise* testing, considers every possible

combination of t input. In this case, more than one pair is derived (i.e., by including higher-level combinations). Pair-wise testing is a specific combinatorial testing technique where test cases are derived by combining values of every pair of an input set. These techniques are also known as orthogonal array testing (OAT).

3.1.5. Decision Table

[1*, c9s6; 1*, c13s6; 4, part 4]

Decision tables (or trees) represent logical relationships between conditions (roughly, inputs) and actions (roughly, outputs). Usually, they are widely adopted for knowledge representation (e.g., machine learning (ML)). Test cases are systematically derived by considering every possible combination of conditions and their corresponding resultant actions. A related technique is *cause-effect graphing*. Shift-left development processes are taking advantage of this kind of testing technique because these techniques are useful for documenting the test results and factors that can affect them.

3.1.6. Cause-Effect Graphing

[1*, c1s6; 4, part 3, part 4]

Cause-effect graphing techniques rely on logical networks that map a set of causes to a set of effects by systematically exploring the possible combinations of input conditions. They identify the effects and link the effects to their causes through model graphs. Cause-effect graphing techniques are used in testing because they allow specification analysis, the identification of the relevant input conditions or causes, the consequent transformations, and the output conditions.

3.1.7. State Transition Testing

[1*, c10; 4, part 4]

Techniques based on Finite-State Machines (*State Transition Testing* techniques in [4, part 4]) focus on representing the SUT with a finite-state machine. In this case, the test

suite is derived to cover the states and transitions according to a specific coverage level.

3.1.8. Scenario-Based Testing

[2*, c8s3, c19s3; 4, part 4; 7]

A model in this context is an abstract (formal) representation of the SUT or its software requirements. (See Modeling in the Software Engineering Models and Methods KA.) *Scenario-based testing* is used to validate requirements, check their consistency, and generate test cases focused on the SUT's behavioral aspects. (See Types of Models in the Software Engineering Models and Methods KA.) The key components of scenario-based testing are the notation used to represent the model of the software or its requirements, workflow models or similar models, the test strategy or algorithm used for test case generation, the supporting infrastructure for the test execution, and the evaluation of test results compared to expected results. Because of the complexity of the techniques, scenario-based testing approaches are often used with test automation harnesses.

Among scenario-based testing, workflow models can also be used to graphically represent the sequence of activities performed by humans and/or software applications. In this case, each sequence of actions constitutes one workflow (also called a scenario). Usually, it is important to ensure that both typical and alternate workflows are also tested. For example, business process testing is part of this scenario-based technique. In this case, the special focus is on the roles in a workflow specification.

3.1.9. Random Testing

[1*, c9s7; 4, part 4]

In this approach, test cases are generated purely at random. This testing falls under the heading of input domain testing because the input domain must be known to be able to pick random points within it. *Random testing* provides a relatively simple approach to test automation. Enhanced forms of random testing (such as adaptive random testing) have been proposed in which other

input selection criteria direct the random input sampling.

Under the name of fuzz testing, the random selection of invalid and unexpected inputs and data is extensively used in cybersecurity to find hackable software bugs, coding errors, and security loopholes. (See also Sections 2.2.8 Security Testing and 8.2 Categories of Tools.)

3.1.10. Evidence-Based

[10, c6s2]

Evidence-based software engineering (EBSE), which follows a rigorous research approach, is the *best* solution for a practical problem. EBSE includes the following phases:

- Identifying the evidence and forming a question
- Tracking down the best evidence to answer the question
- Critically analyzing the evidence in light of the problem that the evidence should help solve.

EBSE principles can also be applied to the testing process. For that purpose, the widely used approaches that allow identifying and aggregating evidence are systematic mapping studies and systematic reviews.

3.1.11. Forcing Exception

[5]

Test cases are specifically conceived for checking whether the SUT can manage a predefined set of exceptions/errors, such as data exception, operation exception, overflow exception, protection exception or underflow exception. Testing techniques usually focus on negative test scenarios (i.e., test cases that are able to force the generation of error messages).

3.2. Structure-Based Test Techniques

[4, part 4]

Structure-based test techniques (sometimes called *code-based test techniques*) focus on the code and its structure. Structure-Based

Test Techniques can be performed at different levels (such as code development, code inspection, or unit testing) and can include static testing (such as code inspection, code walkthrough, and code review), dynamic testing (like statement coverage, branch coverage, and path coverage), or code complexity measurement (e.g., using techniques like cyclomatic complexity [12]).

3.2.1. Control Flow Testing

[1*, c4; 4, part 4]

Control flow testing covers all the statements, branches, decisions, branch conditions, modified condition decision coverage (MC/DC), blocks of statements, or specific combinations of statements in a SUT. The strongest of the control flow-based criteria is path testing, which aims to execute all entry-to-exit control flow paths in a SUT's control flow graph. Because exhaustive path testing is generally not feasible because of loops, other less stringent criteria focus on coverage of paths that limit loop iterations, such as statement coverage, branch coverage, and condition/decision testing. The adequacy of such tests is measured in percentages; for example, when all branches have been executed at least once by the tests, 100% branch coverage has been achieved.

3.2.2. Data Flow Testing

[1*, c5; 4, part 4]

In *data flow testing*, the control flow graph is annotated with information about how the variables are defined, used, and killed (undefined). Commonly adopted data flow testing techniques are All-Definitions Testing, All-C-Uses Testing, All-P-Uses Testing, All-Uses Testing and All-DU-Paths Testing. The strongest data flow testing criterion is the All-DU-Paths Testing, where all definition and use (DU) paths need to be covered [4, part 4]. This is because it requires executing, for each variable, every control flow path segment from a definition of that variable to the use of that definition. However, weaker strategies

such as all-definitions and all-uses are used to reduce the number of paths required.

3.2.3. Reference Models for Structure-Based Test Techniques

[1*, c4]

Although not a technique, a SUT's control structure can be graphically represented using a flow graph to visualize *structure-based test techniques*. A flow graph is a directed graph, the nodes and arcs of which correspond to program elements. (See Graphs and Trees in the Mathematical Foundations KA.) For instance, nodes may represent statements or uninterrupted sequences of statements, and arcs may represent the transfer of control between nodes.

3.3. Experience-Based Techniques

[4, part 1, part 4]

The generation of the most suitable test suite may depend on different factors, such as human knowledge of the SUT and its context and the tester's experience and intuition. In the following section, the commonly adopted experience-based techniques are briefly introduced.

3.3.1. Error Guessing

[1*, c9s8; 4, part 4]

In *error guessing*, software engineers design test cases specifically to anticipate the most plausible faults in each SUT. Good sources of information are the history of faults discovered in earlier projects and the software engineer's expertise.

3.3.2. Exploratory Testing

[4, part 1]

Exploratory testing is defined as simultaneous learning, test design and test execution. The test cases are not defined in advance but are dynamically designed, executed, and modified according to the collected evidence and test results, such as observed product behavior, peculiarities of the SUT, the domain and the

environment, the failure process, the types of possible faults and failures, and the risk associated with a particular product. Usually, the intuition, knowledge, and expertise of the personnel in charge of performing the exploratory testing affect the testing effectiveness. Exploratory testing is widely used in shift-left development (such as Agile). (See Section 5.4.2.)

3.3.3. Further Experience-Based Techniques

[4, part 4; 13]

At the state of the practice, *experience-based techniques* may include further approaches such as Ad Hoc-based, knowledge-based and ML-based testing techniques.

Ad Hoc testing is a widely used technique in which test cases are derived by relying on the software engineer's skill, intuition, and experience with similar programs. It can be useful for identifying test cases that are not easily generated by more formalized techniques. Typical Ad Hoc methodologies are the following:

- Monkey testing runs randomly generated test cases to simulate random activities and cause the program to stop.
- Pair (Buddy) testing involves two individuals. One generates and runs the test cases; the other observes and analyzes the testing process. Pair testing allows for generating test cases with broader and better test coverage.
- Gamification aims to convert testing tasks to components of gameplay. By applying specific techniques (such as engaging practitioners or crowdsourcing complex testing tasks), gamification can substantially improve software testing practice and, consequently, SUT quality.
- Quick testing, in which a very small test suite is selected and executed to swiftly identify critical issues in the SUT. It aims to enhance the probability of detecting faults early in the development process.
- Smoke testing (also known as Build Verification Testing) ensures that the SUT's core functionalities behave

properly. It also guarantees that the SUT is operational before the planned testing begins. In addition, smoke testing prevents failures because of the test environment (e.g., because artifacts or packages are not properly built). Smoke testing is also considered a special case of quick testing.

Knowledge-based testing and ML-based testing exploit (formal or informal) knowledge about the SUT or derive it from observations of SUT executions for defining its behavioral models (such as ontologies or decision tables) (see Section 3.6.1), rules, and non-functional properties. In addition, Knowledge-based testing and ML-based testing specify the testing needs and identify test objectives for which test cases are generated.

3.4. Fault-Based and Mutation Techniques

[1*, c1s14, 1* c3s5; 5]

Fault-based testing techniques devise test cases specifically to reveal likely or predefined fault categories. A *fault model* can be introduced that classifies the different faults to better focus the test case generation or selection. In this context, a variety of platforms and development processes (e.g., waterfall, spiral and Agile) consider the orthogonal defect classification (ODC) a valid methodology for collecting semantic information about the different defects and reducing the time and effort of the root cause analysis.

Mutation Testing was originally conceived as a technique to evaluate test suites (see Section 4.2, Evaluation of the Tests Performed) in which a mutant is a slightly modified version of the SUT (also called *gold*), differing from it by a small syntactic change. Every test case exercises both the gold version and all generated mutants. If a test case succeeds in identifying the difference between the gold version and a mutant, the latter is said to be “killed.” The underlying assumption of mutation testing, the coupling effect, is that more complex

but real faults will be found by looking for simple syntactic faults. For the technique to be effective, many mutants must be automatically generated and executed systematically [6]. Mutation testing is also a testing criterion in itself. Test cases are randomly generated until enough mutants have been killed, or tests are specifically designed to kill surviving mutants. In the latter case, mutation testing can also be categorized as a structure-based technique. Mutation testing has been used effectively for generating fuzz testing. A more recent application of the mutation process is metamorphic testing. This is particularly suitable for in addressing ML systems' testing challenges. In this case, the modifications (called also morph) are applied to the inputs so a relationship can connect the previous input (and its output) to the new morphed input (and its output).

3.5. Usage-Based Techniques

[1*, c15s5]

Usage-based techniques usually rely on a usage model or profiles. In this case, the testing environment needs to represent the actual operational environment, and the sequence of test case execution should reproduce the SUT usage by the target stakeholder. Statistical sampling is used for simulating the execution of many test cases. Thus, sometimes, the term *random testing* is also associated with these techniques. Usage-based statistical testing is applied more during the acceptance testing stage.

3.5.1. Operational Profile

[1*, c15s5, 2*, c11]

Testing based on *operational profiles* aims at generating test cases that estimate the reliability of the SUT or part of it. Therefore, the goal is to infer from the observed test results the future reliability of the software (when it is in use). Because the established reliability strictly depends on the operating profile, the main difficulty (and cost) in using this testing

approach comes from the operational profile derivation. Therefore, one possible solution is to assign to the input the probabilities or profiles according to their frequency of occurrence in actual operation.

3.5.2. User Observation Heuristics

[19*, c5, c7; 4, part 4, annex A]

Specialized heuristics, also called *usability inspection methods*, are applied to systematically observe system use under controlled conditions to determine how well people can use the system and its interfaces. Usability heuristics include cognitive walkthroughs, claims analysis, field observations, thinking aloud, and even indirect approaches such as user questionnaires and interviews.

3.6. Techniques Based on the Nature of the Application

[2* c16, c17, c18, c20, c21; 14*, c4s8; 8]

The above techniques apply to all kinds of software. Additional test derivation and execution techniques are based on the nature of the software being tested. Examples are:

- Object-oriented software
- Component-based software
- Web-based software
- Concurrent programs
- Protocol-based software
- Communication systems
- Real-time systems
- Safety-critical systems
- Service-oriented software
- Open-source software
- Embedded software
- Cloud-based software
- Blockchain-based software
- Big data-based software
- AI/ML/DL-based software
- Mobile apps
- Security and privacy-preserving software

In some cases, standards such as ISO/IEC/IEEE 29119 [4, part 4, part 5] provide examples and support for specifying test cases,

automating their execution, and maintaining the test suites, such as Keyword-Driven Testing [4, part 5].

3.7. *Selecting and Combining Techniques*

[14*, c7s12; 10; 4, part 5]

Combining different testing techniques has always been a well-grounded means to assure the required level of SUT quality. Especially in shift-left developments, methodologies for adaptive combinations of testing techniques are part of the state of the practice. The goal is to improve the effectiveness of testing processes by learning from experience and, at the same time, adapting the technique selection to the current testing session.

3.7.1. *Combining Functional and Structural*

[1*, c9; 4, part 5]

Scenario-based and structure-based test techniques are often contrasted as *functional* vs. *structural* testing. These two approaches to test case selection are seen as complements, as they use different sources of information and highlight different problems. Depending on the different organizational constraints, such as budgetary considerations, they can be combined.

3.7.2. *Deterministic vs. Random*

[1*, c9s6]

Test cases can be selected in a deterministic way, according to many techniques, or randomly drawn from some distribution of inputs, such as is usually done in reliability testing. Several analytical and empirical comparisons have been conducted to analyze the conditions that make one approach more effective than the other.

3.8. *Techniques Based on Derived Knowledge*

[2*, c19, c20; 14*, c7]

Testing techniques can integrate evidence and knowledge from different research areas and contexts. For this, approaches

and methodologies are used to support testing activity and improve its effectiveness. Innovative approaches include using digital twins or simulation methodologies and frameworks, exploiting ML and gamification facilities, and using (simulated) neuronal networks.

4. **Test-Related Measures**

[2*, c24s5; 14*, c10; 4, part 4]

Testing techniques are like tools that help in achieving specific test objectives. To evaluate whether a test objective is reached, well-defined measures are needed. Measurement is usually considered fundamental to quality analysis. Measurement may also be used to optimize test planning and execution. Test management can use several different process measures to monitor progress. (See Software Engineering Measurement in the Software Engineering Management KA for information on measurement programs. See Software Engineering Measurement in the Software Engineering Process KA for information on measures.)

According to the definition in [4, part 4], testing techniques can be classified according to the degree of coverage they can achieve. Coverage may vary from 0% to 100%, excluding possible infeasible tests (i.e., tests that cannot be executed). Thus, for each specification-based, structure-based, and experience-based test technique, the associated coverage measures and the procedure for evaluating that coverage must be determined. Examples of coverage measures could be the percentage of branches covered in the program flow graph or the percentage of functional requirements exercised among those listed in the specifications document.

It is important to consider that monitoring facilities can dynamically compute the ratio between covered elements, and the total number may also be considered. Additionally, especially in the case of structure-based testing techniques, appropriate instrumentation of the SUT may also be necessary.

However, the proposed set of testing measures can also be classified from different

viewpoints — from the point of view of those providing and allowing an evaluation of the SUT based on the observed test outputs and of those that evaluate the thoroughness or effectiveness of the executed test suites.

4.1. *Evaluation of the SUT*

[2*, c24s5]

Usually, indicators (i.e., measurable information) can be used to determine whether a SUT is performing as expected and achieving its expected outcomes. The indicators, sometimes known as key performance indicators (KPIs), are strongly connected with the adopted evaluation measures, methods, data analysis and reporting.

4.1.1. *SUT Measurements That Aid in Planning and Designing Tests*

[14*, c10; 10, c6; 4, part 1, part 4]

All the testing measures proposed in [4, part 4] can be used for planning and guiding testing activities. Additionally, in the shift-left development process, specific measures, such as deployment frequency, lead time, mean time to recovery (MTTR), and change failure rate, are also commonly adopted to plan and manage the testing activities and results.

4.1.2. *Fault Types, Classification and Statistics*

[1* c13s4, c13s5, c13s6]

The testing literature is rich in classifications and taxonomies of faults that can be generic or specific to a context or quality attributes (such as the usability defect classification, the taxonomy of HW/SW security and privacy vulnerabilities and attacks, and the classification of cybersecurity risks). To make testing more effective, it is important to know which types of faults may be found in the SUT and the relative frequency with which these faults have occurred in the past. This information can be useful in making quality predictions and in process improvement (See Characterization in the Software Quality KA).

4.1.3. *Fault Density*

[1*, c13s4; 14*, c10s1]

Traditionally, a SUT can be evaluated by counting discovered faults as the ratio between the number of faults found and the SUT size. Because of the semantics-based definition of faults, additional measurements can be considered, such as fault depth (the minimal number of fault removals needed to make a SUT correct) and fault multiplicity (the number of atomic changes needed to repair a single fault).

4.1.4. *Life Test, Reliability Evaluation*

[1*, c15, 2*, c11; 14*, c1s3]

A statistical estimate of software reliability can be used to evaluate if testing can be stopped or if the SUT is mature enough for the next release. Reliability evaluation is taking a pivotal role in the Cloud (and Fog) contexts [18].

On the one hand, validation and verification proposals are focusing on maintaining the high level of reliability and availability required by the cloud (fog) services. On the other, testing activities are exploiting the computational power of the cloud (fog) environment to speed up the reliability evaluation and drastically reduce its costs.

4.1.5. *Reliability Growth Models*

[1*, c15, 2* c11s5]

Reliability growth models predict reliability based on observed failures. They assume, in general, that when the faults that caused the observed failures have been fixed (although some models also accept imperfect fixes), the product's reliability will increase. There are many published reliability growth models. Notably, these models are divided into *failure-count* and *time-between-failure* models.

4.2. *Evaluation of the Tests Performed*

[4, part 4, c6]

The behavior of the SUT is generally verified by executing test suites, which are pivotal

in finding defects. Therefore, from both the researchers' and practitioners' perspectives, a fundamental part of software testing is comparing test suites. Usually, evaluating the test suites means comparing techniques for test case generation that produce the test cases. Different criteria are used for that purpose, such as coverage criteria or mutation analysis criteria.

4.2.1. Fault Injection

[1*, c2s5]

In *fault injection*, some faults are artificially introduced into the SUT before testing. When a test suite is executed, some of these injected faults are revealed, as are, possibly, some faults that were already there. In theory, depending on which and how many artificial faults are discovered, the testing effectiveness can be evaluated, and the remaining number of genuine faults can be estimated. In practice, statisticians question the distribution and representativeness of injected faults relative to genuine faults and the small sample size on which any extrapolations are based. Some also argue that this technique should be used with great care because inserting faults into the SUT incurs the obvious risk of leaving them there.

4.2.2. Mutation Score

[1*, c3s5; 6]

In mutation testing, the test suite effectiveness measure is calculated as the ratio of killed mutants to the number of generated mutants. The higher the test suite effectiveness value, the better, since it indicates a stronger ability to discover the most real injected faults.

4.2.3. Comparison and Relative Effectiveness of Different Techniques

[1*, c1s7; 5; 9]

Relative effectiveness compares different testing techniques against a specific property, such as the number of tests needed to find the first failure, the ratio of the number of faults found through testing to all the faults found during and after testing, and how much reliability was improved. Several studies have

already been conducted to compare different techniques analytically and empirically according to each notion of property (or effectiveness) defined.

5. Test Process

[4, part 1, part 2, part 3; 2* c8]

Testing concepts, strategies, techniques and measures need to be integrated into a defined and controlled test planning process to test output evaluation. The test process supports testing and provides guidelines to those responsible for different testing activities to ensure the test objectives are met cost-effectively.

As described in [4, part 2], the test process is a multi-layered process activity that includes the test specification at the organizational, management and dynamic levels. The organizational test process defines the steps for creating and maintaining test specifications, such as organizational test policies, strategies, processes, procedures, and other assets [4, part 2].

The test management process defines the steps necessary for management: planning, monitoring and control, and completion.

Finally, the dynamic test process specifies the steps for design and implementation, environment setup and maintenance, execution, and test incident reporting.

In the remainder of this section, some practical considerations about the test process specification, management, and execution, as well as a summary of the test sub-processes and activities included in the organizational, management and dynamic levels as in [4, part 2], are provided.

5.1. Practical Considerations

[4, part 1]

Testing processes should allow the automation of different testing phases and should rely on the controllability, traceability, replicability, and risk/cost estimation of the performed activities. In the remainder of this section, commonly applied test steps are described, compatible with and applicable to all life cycle models. (See Software Life Cycles in the Software Engineering Process KA.)

5.1.1. *Attitudes/Egoless Programming*

[1*, c16; 2*, c3]

An important element of successful testing is a collaborative attitude toward testing and quality assurance (QA) activities. Managers have a key role in fostering a favorable reception toward failure discovery and correction during software development and maintenance. For instance, in shift-left change in development, such as Agile, communication and collaboration among testers and developers are considered vital for achieving successful testing results.

5.1.2. *Test Guides and Organizational Process*

[1*, c12s1, 2* c8; 4, part 2, part 3; 14*, c7s3]

Various aims can guide the testing phases. For example, risk-based testing uses the product risks to prioritize and focus the test strategy, and scenario-based testing defines test cases based on specified software scenarios and backlog lists. Usually, the organization of the test process includes defining test policies (i.e., specifying the purpose, goals, and overall scope of testing) and test strategies (i.e., specifying the guidelines about how testing will be carried out). For instance, in shift-left developments, a test strategy should include at least the following data: the purposes (e.g., defined through user stories), the objectives (e.g., a test suite), the scope (the SUT), and the environment and methods (e.g., how, and where the test suite is run).

5.1.3. *Test Management and Dynamic Test Processes*

[1*, c12; 4, part 2, part 3, 14*, c7s3]

Test activities conducted at different levels (see Section 2, Test Levels) should be organized — with people, tools, policies, and measures — into a well-defined process integral to the life cycle. Test process management includes different subprocesses such as planning, monitoring, control, and completion, whereas the dynamic test process includes test design and

implementation, test environment set-up and maintenance, test execution, and test incident reporting.

5.1.4. *Test Documentation*

[1*, c8s12; 14*, c7s8; 4, part 3]

According to [4, part 3], documentation is integral to the formalization of the test process. Test documents can be classified into three hierarchical categories: organizational test documentation, test management documentation and dynamic test documentation. Organizational test documentation includes the information necessary for documenting the test policy and the organizational test strategies. Test management documentation includes the test plan, test status report and test completion report. Finally, dynamic test documentation includes the following documents: test specification (test design specification, test case specification and test procedure specification), test data requirements, test environment requirements, test data readiness report, test environment readiness report, and test execution documentation (such as actual results, test results, test execution log and incident report).

Test documentation should be produced and continuously updated with the same quality as other software engineering documentation. Test documentation should also be under the control of software configuration management. (See the Software Configuration Management KA.)

5.1.5. *Test Team*

[1*, c16; 2* c23s5; 4, part 2, part 3]

Formalizing the testing process may also involve formalizing the testing team's organization. Considerations of cost, schedule, maturity levels of the involved organizations and criticality of the application can guide the decision. The testing team can be composed of members involved (or not) in the SUT development (i.e., having or not having an unbiased, independent perspective) or internal (or external) personnel. Shift-left development

does not strongly distinguish among testing team members because the test suite is defined and updated according to the SUT development and delivered code.

5.1.6. *Test Process Measures*

[1*, c18s3, 14*, c10; 4, part 1, part 2, part 3]

Managers use several measures for the resources spent on testing, as well as for the relative fault-finding effectiveness of the various test phases, to control and improve the testing process, as well as to provide information for managing process risks. Therefore, monitor and control testing must define required data and information and state how to obtain them. The test measures may cover the number of specified, executed, passed, and failed test cases, among other elements. These measures can also be combined with specific process metrics such as residual risk, cumulative defects open and closed, test case progress, and defect detection percentage. Evaluation of test phase reports can be combined with root-cause analysis to evaluate test process effectiveness in finding faults as early as possible. Such an evaluation can be associated with risk analysis. Moreover, the resources deemed worth spending on testing should be commensurate with the application's use and criticality. Different techniques have different costs and yield different confidence levels in product reliability.

5.1.7. *Test Monitoring and Control*

[4, part 1, part 2]

Monitoring and Control comprise an important sub-process of the test management process as in [4, part 2], useful for collecting data and information required during test management and assessment. Usually, monitoring and control activities are executed in parallel with the test execution, and sometimes, data collected might prompt revision of overall process planning. Monitoring assures that testing process activities comply with a specific test plan to trace the requirements satisfaction

and mitigate the identified risks satisfactorily. During test monitoring and control, specific documentation (test reports) can regularly be produced to help assess and document the test activity.

5.1.8. *Test Completion*

[14*, c7s11; 4, part 3]

A decision must be made about how much testing is enough and when a test stage can be completed. Therefore, the purpose of *Test Completion*, a sub-process of the test management process as in [4, part 2], is to ensure that test requirements are satisfied and verified, test reports are completed, and test results are communicated to relevant stakeholders. Thoroughness measures, such as achieved code coverage or functional coverage, and estimates of fault density or operational reliability, provide useful support but are not sufficient in themselves. The decision also involves considerations about the costs and risks incurred by possible remaining failures, as opposed to the costs incurred by continuing to test (See Test Selection and Adequacy Criteria in Section 1.2, Key Issues.) As for the other activities, in this stage, specific documentation is produced (e.g., test completion report) and communicated to the relevant stakeholders.

5.1.9. *Test Reusability*

[14*, c3; 9]

It is necessary to add complexity and time for test planning and design to achieve reusability of the testing artifacts, such as the test case or execution environment, which is desired when test development is costly, time-consuming, and complex.

Test reusability collects and classifies the testing knowledge (test cases and test results) to make this information searchable and usable for creating new tests or re-executing an existing one. Suitable knowledge-based repositories should be configured and managed to test reusability so changes to software requirements or design can be reflected in changes to the tests.

The reusability of test cases is pivotal in feature-based or product-line development and regression testing. Test reusability also relates to maintainability because reusability can reduce the cost and effort involved and improve a test's effectiveness.

5.2. Test Sub-Processes and Activities

[1*, c1s12; 1*, c12s9; 4, part 2]

In the remainder of this section, the main testing activities and sub-processes are briefly introduced.

5.2.1. Test Planning Process

[1*, c12s1, c12s8; 11; 4, part 2]

Like all other aspects of project management, testing activities must be planned. According to [4, part 2], key aspects of test planning include identification and coordination of personnel, identification of the test objective and completion criteria, definition of test facilities and equipment, creation and maintenance of all test-related documentation, and risk planning and management for possible undesirable outcomes. These activities can be organized at three different levels: (1) process management (i.e., identification of test policies, strategies, processes, and procedures), (2) organizational management (i.e., definition of the test phase, test type and test objective), and (3) design and implementation (i.e., definition of the test environment, the test execution process and monitoring, the completion process, and reporting).

5.2.2. Test Design and Implementation

[1*, c12s1, c12s3; 11]

Generation of test cases is based on the level of testing to be performed and the chosen testing techniques. According to the dynamic test process, as described in [4, part 2], preconditions of the test case generation are the identification of test objectives and the selection of the appropriate testing/demonstration techniques. Test generation

focuses on implementing and executing test cases. It often relates to tooling (i.e., using specific software, also called a *test cases generator*). This software accepts inputs (such as source code, test criteria, specifications, or data structure definitions) and uses them to generate the test suites. Sometimes, a test case generator can determine expected results by using a specific oracle facility. This contributes to the full test automation of the overall testing process.

5.2.3. Test Environment Set-up and Maintenance

[1*, c12s6; 2* c8s1; 14* c13s2; 4, part 2; 11]

According to the dynamic test process, as described in [4, part 2], test environment development and setup involve identifying the testing infrastructure. This includes selecting or developing the facilities, hardware, software, firmware, and procedures to conduct the testing activity. The testing environment can be simulated, controlled, and executed in vitro or in vivo. Developing the test environment also involves setting up monitoring and logging facilities useful for documenting the testing activities and assessing the result obtained. The testing environment should be compatible with the other software engineering tools used.

5.2.4. Controlled Experiments and Test Execution

[1*, c12s7, 14* c4s7, 14* c5s6; 4, part 2]

Execution of tests should embody a basic principle of scientific controlled experimentation — everything done during testing should be performed and documented specifically and clearly enough that another person could replicate the results. Hence, testing should be performed following documented procedures using a clearly defined version of the SUT. Especially during acceptance testing, controlled experiments like A/B testing can also be performed to statistically evaluate user preferences between different versions of the SUT.

5.2.5. Test Incident Reporting

[1*, c13s4, c13s9, c13s11; 2*, c8s3; 14*, c7s8; 4, part 3; 12]

According to the dynamic test process, as described in [4, part 2], testing incidents and reporting focus on the well-defined test data collection process (i.e., identifying when a test was conducted, who performed the test, what software configuration was used, and other relevant identification information). This process and the collected evidence can be leveraged for accountability purposes. Test reporting can involve suitable audit systems to identify unexpected or incorrect test results and record them in a problem reporting system. These data form the basis for later debugging and fixing the problems observed as failures during testing. Also, anomalies not classified as faults could be documented if they later become more serious than first thought. Test reports are also inputs to the change management request process. (See Software Configuration Control in the Software Configuration Management KA.)

Hence, the *Test Incident Reporting* process focuses on identifying the relevant stakeholders' incidents that could be used to determine what aspects of software testing and other processes need improvement and how effective previous approaches have been.

Part of the incident reporting is also evaluating test results to determine whether the testing has been successful. In most cases, "successful" means that the software performed as expected and did not have any major unexpected outcomes. Not all unexpected outcomes are necessarily faults; sometimes they are determined to be simply noise. Before a fault can be removed, an analysis and debugging effort is needed to isolate, identify, and describe it. When test results are particularly important, a formal review board may be convened to evaluate them.

5.3. Staffing

[1*, c16; 4, part 3]

According to [4, part 3], staffing includes defining roles, activities, and responsibilities,

specifying hiring needs, and defining training needs. Staffing affects project risk because the team's expertise might undermine the ability to discover faults, to address changing requirements, to meet deadlines, and increase/reduce maintenance costs.

The roles, activities and responsibilities definition establishes the following roles and responsibilities: the activity leader and supporting personnel, the test-related roles and their corresponding responsibilities, and the person responsible for providing the test item(s).

Depending on the development lifecycle adopted, typical testing roles include but are not limited to scrum master/test lead, QA/test analyst, test designer, test security/performance engineer and consultant, test environment expert, test executor and test automation consultant or architect.

Hiring needs require the identification of specific requirements for which additional testing personnel are needed to complete the testing process (as well as when that personnel is needed and the desired skills). Depending on the business needs, staffing could take different forms, from internal transfers to external hires or even consultants and/or outsourced resources.

Finally, the training needs specification includes the definition of the required skill level. It also includes the specification of the training activities (such as classroom training, self-paced training, computer-based training, or mentoring) useful for providing the necessary skills to the selected staff.

6. Software Testing in the Development Processes and the Application Domains

[2*, c8, c15; 14*, c4s8, c7]

Whatever development process is adopted, testing remains a fundamental activity. However, specific testing activities or terminologies could be used in some cases, such as the adopted development life cycle and/or the application domain.

6.1. *Testing Inside Software Development Processes* [2*, c8; 14*, c7]

In the remainder of this section, peculiarities of testing inside the different development processes are provided.

6.1.1. *Testing in Traditional Processes* [1* c18; 14*, c7]

There are a variety of traditional processes, essentially based on the SUT development principles, that can be adopted within the organization. Sequential, V, spiral model and iterative are just some of the processes commonly applied. (Software Life Cycles in the Software Engineering Process KA provides a detailed description of each.) However, in all these processes, testing is just one perceived activity; it is sometimes performed at the end of the process, with a tangible risk of SUT development failure in case of deviation of the end-user needs or assessment issues. During recent years, to evaluate and control the overall quality of the SUT, initiatives such as test maturity model integration (TMMi) and software process improvement (SPI) have been established. As a result, different existing frameworks have been updated or improved for the purpose, such as software process improvement and capability determination (SPICE), capability maturity model integration (CMMI), and unified process (UP).

For instance, CMMI is one of the most referenced models; it can guide key SUT stakeholders in gaining control of their development and maintenance processes. It is, in fact, a well-defined set of best practices in software testing that improves SUT quality by increasing customer satisfaction.

Presented in the early 2000s, the UP model can be seen as a predecessor of the shift-left movement. UP encourages testing early by offering several mechanisms to integrate testing more closely with the software development effort, making testing a distinct discipline. Furthermore, UP promotes an iterative development approach for continuously

verifying quality. It also enables use cases and risk to drive SUT development and allows strategic change management. UP groups the SUT increments and SUT iterations into four phases: inception, elaboration, construction, and transition.

UP can be considered both iterative and Agile — Iterative because all the core activities are repeated throughout the SUT development project, and Agile because the defined phases of the chosen lifecycle can be repeated until the SUT meets requirements (both functional and non-functional), achieves the defined objectives, and guarantees the target quality.

6.1.2. *Testing in Line with Shift-Left Movement* [2*, c3, c8s2; 4, part 1; 10, c3, c5]

The *shift-left testing movement* promotes the adoption of testing in the early stages of software development to detect and remove faults as early as possible to increase overall SUT quality and reduce the cost and risks of testing activities. Different development life cycles, such as Agile, DevOps and TDD, belong to the shift-left movement. (See Agile Methods in the Software Engineering Process KA.)

In shift-left-based development, different testing aspects should be considered:

- A. The internal code quality: Regression, prioritization, security, and privacy could be the primary objectives of the internal code quality (Section 2.2). Usually, unit testing and integration testing are the targeted levels (Section 2.1), whereas structure-based is the main testing technique (Section 3.2).
- B. Business needs: Compliance and conformance, usability, security, and privacy are just a subset of the possible objectives of the business needs aspect (Section 2.2). Concerning this aspect, testing focuses more on the system and acceptance test levels and on end-user expectations, as well as usage-based (Section 3.5) and scenario-based techniques (Section 3.1.8).

- C. Perceived quality: Alpha, beta, installation, usability, security, and privacy could be the primary objectives of the internal perceived quality (Section 2.2). Perceived quality usually focuses on the acceptance test level and is achieved by applying techniques based on software engineering's intuition and experience (Section 3.3) and usage-based and fault-based techniques, such as mutation testing (Section 3.4).
- D. Quality assurance: Performance installations, security, and privacy conformance and compliance are some main objectives of quality assurance (Section 2.2). This aspect may involve all testing levels, and the selection of the testing technique depends on the objective and the level chosen.

Examples of testing inside the different shift-left movements implementation are:

- In Agile process development, testing activities involve all stakeholders (such as customers and team personnel) and target the identification of where improvements could be made in future interactions. Managing the risk of regression defects, meeting changing requirements, and managing their impact on test artifacts are also objectives of the Agile testing process. Typically, test automation is used to manage the regression risk, and exploratory testing may be used to manage a lack of detailed requirements.
- In TDD, the test cases mainly target the software requirements specifications and acceptance, and they are generated in advance of the code being written. The tests are based on the user stories and implemented using automated component testing tools. TDD is a practice that requires defining and maintaining unit tests and can help clarify the user needs and software requirements specifications.

- In testing automated builds and continuous integration (for instance, DevOps), the SUT is continuously developed, integrated, delivered and monitored. In this process, regression testing is continuously performed to timely identify and correct development and integration issues. Additionally, quick testing techniques, such as smoke testing, are commonly used during continuous integration to guarantee that the SUT is testable before it is released to the operational stage.

6.2. Testing in the Application Domains

[2*, c15; 14*, c4s8]

Usually, an application domain is strictly connected to a certain reality. Therefore, testing approaches could be tailored to the needs of the domain and customized to the adopted technologies.

Each domain-specific environment has specific aspects and solutions for software testing such as:

- Automotive domain testing: Due to the complexity of automotive systems, this testing involves aspects of almost every software component and its interaction with hardware. Security testing, simulation testing, reliability/life cycle testing, integrated systems testing, data acquisition and signal analysis testing, quality testing and inspection, and stress/strain testing are just some of the various testing performed in this domain. Several supporting standards guide and manage automotive testing according to the peculiarity, the component, or the quality aspect that should be assessed. Autosar² and Automotive SPICE³ are examples.
- Internet of things (IoT) domain testing: This testing involves application development, device management, system management, heterogeneity management, data management, and tools for analysis,

² <https://www.autosar.org/>

³ <https://www.automotivespice.com/>

deployment, monitoring, visualization and research. Additionally, security, privacy, communications and user/component interaction should be considered in the quality assessment. For example, guidelines and specific conformance test suites for cybersecurity assessment of the IoT SUT are detailed in the European Telecommunications Standards Institute (ETSI) standards.⁴

- Legal domain testing: One of the most important aspects in the legal domain is the management of highly sensitive users; therefore, security, privacy and trust are the most common areas of focus for testing. Additionally, because of the copious data collected and exchanged, performance testing of the data repository, testing to show accurate communication and integration testing, as well as consistency and compliance testing, should also be done. Finally, because the legal domain is characterized by specific nomenclature and jargon, involving legal domain experts in test case generation is common practice to ensure a focus on desired characteristics and quality.
- Mobile domain testing: This testing is usually for usability, functional, configuration and consistency assessment. Mobile-specific aspects such as screen resolution, global positioning system (GPS), operating systems, and device manufacturers should also be considered during testing activity. Finally, the type of mobile applications (native or web apps) and their interactions need to be tested. For example, the W3C Web and Mobile Interest Group⁵ provides facilities, guidelines and ad hoc test suites useful for developing and testing web-based content, applications and services.
- Avionics domain testing⁶: Usually, avionics systems include several independent or loosely coupled components and commercial off-the-shelf products. Therefore testing needs to include very general processes and approaches applicable at both the system and the process levels. Functional and non-functional, integration, communication operational, stress, safety, and security testing are examples of possible approaches. As in other domains, supporting standards such as Aeronautical Radio Incorporated (ARINC) Standards and ASTM F3153-15 can be used for reference.
- Healthcare domain testing: Healthcare domain testing should ensure quality in areas such as secure and reliable data exchange, stable performance, privacy, and safety. Interoperability, usability, performance and compliance with industry regulations, as well as security and safety standards (such as the Health Level Seven (HL7),⁷ Fast Healthcare Interoperability Resources (FHIR),⁸ Digital Imaging and Communications in Medicine (DICOM),⁹ Health Insurance Portability and Accountability Act (HIPAA),¹⁰ and the General Data Protection Regulation (GDPR)¹¹) should also be considered.
- Embedded domain testing: Because software and hardware are tightly coupled in embedded systems, testing activity should assess functional and non-functional attributes of both software and hardware.
- Graphical user interface (GUI) testing: GUI testing involves assessing the UI (user interface) (i.e., the elements of the

4 <https://www.etsi.org/>

5 <https://www.w3.org/2013/07/webmobile-ig-charter.html>

6 www.astm.org.

7 <https://www.hl7.org/>

8 <https://fhir.org/>

9 <https://www.dicomstandard.org/>

10 <https://www.hhs.gov/hipaa/>.

11 <https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0679>.

user objects that we can see). Thus, GUI testing targets the design pattern, images, alignment, spellings, and the overall look and feel of the UI. Testing approaches based on finite-state machines, goal-driven approaches, approaches based on abstractions and model-based approaches can be considered.

- **Gaming:** Gaming applications and software are causing increased demand for new approaches and ways to ensure their quality and security. Among the specific testing techniques, playtesting is one of the most adopted. In this case, real gamers (usually from the development of testing teams) repeat quality control methods at many points of the game execution or design process. GUI testing, functionality testing, security testing, console testing, compliance testing and performance testing can also be considered.
- **Real-time domain testing:** Real-time testing usually focuses on assessing timing constraints and deterministic behavior. Unit, integration and system testing approaches can be adopted. Communication, interaction and behavioral testing can also be performed.
- **Service oriented architecture (SOA) testing:** This testing focuses mainly on correctly implementing business processes and involves unit and integration testing approaches. Structure-based, specification-based and security testing can be applied. The testing activity varies according to the environment, organization and set of requirements that should be satisfied.
- **Finance domain testing:** This testing covers a wide range of aspects, from managing financial requirements to assessing financial applications and software programs. As in other domains, domain-specific knowledge (such as that held by, for example, banks, credit unions, insurance companies, credit card companies, consumer finance businesses, investment funds and stock brokerages) could be necessary to apply the testing process

effectively and efficiently. Customer satisfaction, usability, security, privacy, third-party component and apps integrations, real-time issues, and performance are some of the most important challenges in this domain.

7. Testing of and Testing Through Emerging Technologies

Software development was driven by emerging trends such as the widespread diffusion of mobile technology, cloud infrastructures adoption, big data analysis and the software as a service paradigm, which highlighted new constraints and challenges for testing.

7.1. Testing of Emerging Technologies

- **Testing artificial intelligence (AI), ML/ deep learning (DL) [13]:** AI, ML and DL are being applied in practice. Most business applications will have some form of AI, ML or DL. Because of their peculiarities (for instance their non-deterministic nature), testing such applications is challenging and might be very expensive. Three main aspects should be considered in defining bugs and testing in this scenario: the required conditions (correctness, robustness, security, and privacy); the AI, ML or DL items (e.g., a bug might exist in the data, the learning program, or the framework used); and the involved testing activities (test case generation, test oracle identification and definition, and test case adequacy criteria). In all these applications, a prototype model is first generated based on historical data. Then, offline testing, such as cross-validation, is conducted to verify that the generated model satisfies the required conditions. Usually, after deployment, the model is used for prediction purposes by generating new data. Finally, the generated data is analyzed through online testing to evaluate how the model interacts with user behaviors.

- Testing blockchain [15]: The commonly used testing techniques for validating blockchains and related applications such as smart contracts are stress testing, penetration testing and property testing. However, depending on the specific situation, different aspects should be considered during the testing of a blockchain-based SUT, such as the following:
 - Platform type: The level of validation depends on the type of platform used for implementation — public or private. The latter requires a much greater testing effort.
 - Connection with other applications: Integration testing should be performed to check consistency when the blockchain works with various applications.
 - Performance: Specific strategies to handle many transactions should be conceived to guarantee a satisfactory performance level. Qualitative and quantitative metrics, such as average transaction validation latency and security, should also be considered.
- Testing the cloud [1*, c10s10, 2*, c18]: Testing the cloud validates the quality of applications and infrastructures deployed in the cloud by considering both functional and non-functional properties. The focus is to identify problems posed by systems residing in the cloud. Therefore, testing activities use techniques to validate cloud-based services' performance, scalability, elasticity and security. Moreover, testing should also focus on compatibility and interoperability among heterogeneous cloud resources when different deployment models are used (e.g., private, public or hybrid).
- Testing concurrent and distributed applications [1*, c10s10, 2*, c17]: One main aspect of testing dynamic, complex, distributed or concurrent applications is dealing with multiple operating systems and updates, multiple browser platforms

and versions, different types of hardware, and many users. For such testing, it's difficult to use testing approaches based on the classical hierarchy between components or systems; instead, solutions based on input/output, dependency threads, or dynamic relations often work better. Additionally, the possibility of continuous integration and deployment of the different components forces the testing process to include approaches for managing continuous test operation, injection, monitoring and reporting according to the time, bandwidth usage, throughput, and adaptability constraints. Finally, there is still the need for solutions that allow the reusability of testing knowledge, architectures, and code to make the testing activity more effective and less expensive.

7.2. Testing Through Emerging Technologies

- Testing through ML [13]: AI, ML or DL techniques are successfully used to reduce the effort involved in several activities in software engineering (such as behavior extraction, testing or bug fixing). These techniques aid both researchers and practitioners in adopting and identifying appropriate methods for their desired applications. There is a growing interest in adopting ML techniques in software testing because most software testing issues are being formulated as ML learning problems. Indeed, AI, ML or DL is used in almost all software, such as test case design, the oracle problem, test case evaluation, test case prioritization and refinement, and mutation testing automation. Indeed, they reduce maintenance efforts and improve the overall SUT quality because of their ability to analyze large amounts of data for classifying, triaging and prioritizing bugs more efficiently. From a DevOps perspective, AI, ML and DL solutions can be used in SUT automation authoring and execution phases of test cases, as well as in the post-execution test analysis that

identifies trends, patterns and impact on SUT testing activity.

- Testing through blockchain [15]: Testing becomes complicated when different teams, domain experts and users need to work together in collaborative, large-scale systems and complex software systems to achieve a common goal. This is mainly because of the time constraint, data sharing policies, acceptance criteria and trusted coordination among the teams involved in the testing process. Blockchain technologies can be exploited to improve software testing efficiency and avoid using centralized authority to manage different testing activities. This help ensure distributed data management, tamper resistance, auditability, and automatic requirement compliance to improve the quality of software testing and development. Blockchain-based approaches for trusted test case repository management and to support test-based software and security testing are also considered.
- Testing through the cloud [17]: Testing through the cloud refers to SUT testing performed by leveraging scalable cloud technologies. Usually, the cloud is used for testing purposes wherever large-scale simulations and elastic resources are necessary. Indeed, this can affect cost reduction, development, and maintenance of the testing infrastructure (scaffolding), and online validation of systems, such as ML-based SUT. A particular situation is the testing of the cloud through the cloud itself. This is an example of the intersection between testing of and testing through emerging technologies. The applications and infrastructures deployed in the cloud can be tested, exploiting the cloud's bandwidth.
- Testing through simulation [1*, c3s9]: Simulation is an important technology for testing activity because it represents a valid means for evaluating SUT execution under critical situations or disasters or assessing specific behaviors or recovering activities. The complexity of the

testing approach might vary according to the complexity of the simulation system adopted and might involve closed-loop testing; assessing the devices, communications, and interface; and use of real-time data (e.g., voltage, current and breaker status). Simulation testing can be applied to each development level and might involve mathematical, formal representation of the real system, environment, network conditions and control devices. Simulation testing is currently adopted in many application domains. Especially in the automotive and embedded domain, among the different proposals, one of the emerging solutions for simulation testing is hardware-in-the-loop (HIL) simulation testing. In this case, real signals sent to the SUT to simulate reality and to test and design the iteration are continuously performed while the real-world system is being used.

- Testing through crowdsourcing [16]: Crowdsourced testing (also known as crowdtesting) is an approach for involving users and experts in the testing activity. Thus, crowdsourcing users represent the dispersed, temporary workforce of multiple individual testers. Testing through crowdsourcing is mainly used for testing mobile applications because it ensures technology diversity and customer-centric validation. However, crowdtesting is not a substitute for in-house SUT validation. It represents a valid means of detecting failures and issues because it involves many individuals (testers) in different locations, who are using different technologies in different conditions and who have different skills and knowledge.

8. Software Testing Tools

[1*, c12s11, 14*, c7]

Several testing tools focus on the SUT peculiarities and needs. This section describes the main issues and challenges concerning testing tools and categorizes them.

8.1. Testing Tool Support and Selection

[1*, c12s11, 14*, c7]

Testing involves many labor-intensive tasks since it involves running numerous program executions and handling a considerable amount of information. Appropriate tools can alleviate the burden of tedious clerical operations and make them less error-prone. Sophisticated tools can support test design and generation, making them more effective.

Guidance for managers and testers on selecting testing tools is crucial, as the right tool significantly impacts testing efficiency and effectiveness. Tool selection depends on diverse factors, such as development choices, evaluation objectives and execution facilities. In general, there might not be a unique tool to satisfy specific needs, so a suite of selected tools could be appropriate.

8.2. Categories of Tools

[1*, c1, c3, c4, c7, c8, c9, c12]

Several classifications of testing tools mainly describe their functionalities, such as the following:

- *Test harnesses* (drivers, stubs) [1*, c3s9] provide a controlled environment in which tests can be launched and the test outputs can be logged. Drivers and stubs are provided to execute parts of a SUT to simulate calling and called modules.
- *Test generators* [1*, c12s11] assist in generating test cases. That generation can be random, path-based, model-based or a mix thereof.
- *Capture/replay tools* [1*, c12s11] automatically re-execute or replay previously executed tests that have recorded inputs and outputs (e.g., screens).
- *Oracle, file comparators, assertion checking tools* [1*, c9s7] assist in deciding whether a test outcome is successful.
- *Coverage analyzers and instrumenters* [1*, c4] work together. Coverage analyzers assess which and how many entities of

the program flow graph have been exercised among all those required by the selected test coverage criterion. The analysis can be done through SUT instrumenters that insert recording probes into the code.

- *Tracers* [1*, c1s7] record the history of a program's execution paths.
- *Regression testing tools* [1*, c12s16] support the re-execution of a test suite after a section of software has been modified. They can also help select a test subset according to the change made.
- *Reliability evaluation tools* [1*, c8] support test results analysis and graphical visualization to assess reliability-related measures according to selected models.
- *Injection-based tools* [1*, c3, c7s7] focus on introducing or reproducing specific problems to confirm that the SUT behaves suitably under the corresponding condition. That can involve managing some input or triggering of events. Usually, two categories of injection-based tools are considered: attack injection and fault injection.
- *Simulation-based tools* [1*, c3s9] verify and validate selected properties. Usually, they exploit specific models to enable the automated execution of scenarios to assess whether the SUT operates as expected or to predict how the SUT would respond to defined inputs. Typical simulation-based tools are classified into tools for verification, tools for collaboration, tools for optimization, tools for testing automated systems and tools for evaluating software concepts.
- *Security testing tools* [1*, c8s3, c12s11] focus on specific security vulnerabilities. Among these are tools for attack injection, penetration testing and fuzz testing.
- *Test management tools* [1*, c12s11] include all the supporting tools that assure efficient and effective test management and data collection.
- *Cross-browser testing tools* [1*, c8s3] enable the tester to quickly build and run user interface test cases across desktop, mobile

and web applications to check whether the SUT looks and works as expected on every device and browser.

- *Load testing tools* [1*, c3] collect valuable data and evidence for SUT performance evaluations.
- *Defect tracking tools* [1*, c3] help keep track of detected faults during the SUT development projects. These tools behave as tracking systems and allow end users to enter fault reports directly.
- *Mobile testing tools* [1*, c8s3] support the implementation and testing of mobile apps by allowing several repeated UI tests over the application platform, development on real mobile devices or emulators, testing of the mobile apps on real-time

implementations and collection of data for specific QA measures.

- *API testing tools* [1*, c7s2] check whether the applications meet functionality, performance, reliability, and security expectations throughout the automation of specific API tests.
- *Web application testing tools* [1*, c8s3], also referred to as web testing tools, support validating the functionality and the performance of web-based SUTs. These tools provide relevant insight and data for different stakeholders, such as developers, server managers, and infrastructure administrators. These tools address issues, or bugs before SUTs are available to end users.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	1*	2*	14*	19*
1. Software Testing Fundamentals	c1, c2	c8	c7	
<i>1.1. Faults vs. Failures</i>	c1s5	c1	c1s3	
<i>1.2. Key Issues</i>				
<i>1.2.1. Test Case Creation</i>	c12s1, c12s3	c8		
<i>1.2.2. Test Selection and Adequacy Criteria</i>	c1s14, c6s6, c12s7	c8		
<i>1.2.3. Prioritization/Minimization</i>				
<i>1.2.4. Purpose of Testing</i>	c13s11, c11s4	c8		
<i>1.2.5. Assessment and Certification</i>		c7, c25		
<i>1.2.6. Testing for Quality Improvement/Assurance</i>	c16s2			
<i>1.2.7. The Oracle Problem</i>	c1s9, c9s7			
<i>1.2.8. Theoretical and Practical Limitations</i>	c2s7			
<i>1.2.9. The Problem of Infeasible Paths</i>	c4s7			
<i>1.2.10. Testability</i>	c17s2			
<i>1.2.11. Test Execution and Automation</i>				
<i>1.2.12. Scalability</i>	c8s7			
<i>1.2.13. Test Effectiveness</i>	c1s1	c8s1		
<i>1.2.14. Controllability, Replication and Generalization</i>	c12s12			
<i>1.2.15. Offline vs. Online Testing</i>				

<i>1.3. Relationship of Testing to Other Activities</i>				
2. Test Levels	c1s13	c8s1		
<i>2.1. The Target of the Test</i>	c1s13	c8s1		
<i>2.1.1. Unit Testing</i>	c3	c8		
<i>2.1.2. Integration Testing</i>	c7	c8		
<i>2.1.3. System Testing</i>	c8	c8		
<i>2.1.4. Acceptance Testing</i>	c1s7	c8s4		
<i>2.2. Objectives of Testing</i>	c1s7			
<i>2.2.1. Conformance Testing</i>	c10s4			
<i>2.2.2. Compliance Testing</i>	c12s3			
<i>2.2.3. Installation Testing</i>	c12s2			
<i>2.2.4. Alpha and Beta Testing</i>	c13s7, c16s6	c8s4		
<i>2.2.5. Regression Testing</i>	c8s11, c13s3			
<i>2.2.6. Prioritization Testing</i>	c12s7			
<i>2.2.7. Non-functional testing</i>	c8s7, c8s8, c14s2, c15, c17s2	c8, c 11, c17		
<i>2.2.8. Security Testing</i>		c13		
<i>2.2.9. Privacy Testing</i>		c13, c14		
<i>2.2.10. Interface and API Testing</i>		c8s1	c7s12	
<i>2.2.11. Configuration Testing</i>	c8s5			
<i>2.2.12. Usability and Human-Computer Interaction Testing</i>		c8s4		c6
3. Test Techniques	c1s15			
<i>3.1. Specification-Based Techniques</i>	c6s2			
<i>3.1.1. Equivalence Partitioning</i>	c9s4			
<i>3.1.2. Boundary Value Analysis</i>	c9s5			
<i>3.1.3. Syntax Testing</i>	c10s11	c5		
<i>3.1.4. Combinatorial Test Techniques</i>	c9s3			
<i>3.1.5. Decision Table</i>	c9s6, c13s6			
<i>3.1.6. Cause-Effect Graphing</i>	c1s6			
<i>3.1.7. State Transition Testing</i>	c10			
<i>3.1.8. Scenario Testing</i>		c8s3.2, c19s3.1		
<i>3.1.9. Random Testing</i>	c9s7			
<i>3.1.10. Evidence-Based</i>				
<i>3.1.11. Forcing Exception</i>				
<i>3.2. Structure-Based Test Techniques</i>				
<i>3.2.1. Control Flow Testing</i>	c4			
<i>3.2.2. Data Flow Testing</i>	c5			

3.2.3. <i>Reference Models for Structure-Based Test Techniques</i>	c4			
3.3. <i>Experience-Based Techniques</i>				
3.3.1. <i>Error Guessing</i>	c9s8			
3.3.2. <i>Exploratory Testing</i>				
3.3.3. <i>Further Experience-Based Techniques</i>				
3.4. <i>Fault-Based and Mutation Techniques</i>	c1s14, c3s5			
3.5. <i>Usage-Based Techniques</i>	c15s5			
3.5.1. <i>Operational Profile</i>	c15s5	c11		
3.5.2. <i>User Observation Heuristics</i>				c5, c7
3.6. <i>Techniques Based on the Nature of the Application</i>		c16, c17, c18, c20, c21	c4s8	
3.7. <i>Selecting and Combining Techniques</i>			c7s12	
3.7.1. <i>Combining Functional and Structural</i>	c9			
3.7.2. <i>Deterministic vs. Random</i>	c9s6			
3.8. <i>Techniques Based on Derived Knowledge</i>		c19, c20	c7	
4. Test-Related Measures		c24s5	c10	
4.1. <i>Evaluation of the SUT</i>		c24s5		
4.1.1. <i>SUT Measurements That Aid in Planning and Designing Tests</i>			c10	
4.1.2. <i>Fault Types, Classification and Statistics</i>	c13s4, c13s5, c13s6			
4.1.3. <i>Fault Density</i>	c13s4		c10s1	
4.1.4. <i>Life Test, Reliability Evaluation</i>	c15	c11	c1s3	
4.1.5. <i>Reliability Growth Models</i>	c15	c11s5		
4.2. <i>Evaluation of the Tests Performed</i>				
4.2.1. <i>Fault Injection</i>	c2s5			
4.2.2. <i>Mutation Score</i>	c3s5			
4.2.3. <i>Comparison and Relative Effectiveness of Different Techniques</i>	c1s7			
5. Test Process		c8		
5.1. <i>Practical Considerations</i>				
5.1.1. <i>Attitudes/Egoless Programming</i>	c16	c3		
5.1.2. <i>Test Guides and Organizational Process</i>	c12s1	c8	c7s3	
5.1.3. <i>Test Management and Dynamic Test Processes</i>	c12		c7s3	
5.1.4. <i>Test Documentation</i>	c8s12		c7s8	
5.1.5. <i>Test Team</i>	c16	c23s5		
5.1.6. <i>Test Process Measures</i>	c18s3		c10	
5.1.7. <i>Test Monitoring and Control</i>				

5.1.8. Test Completion			c7s11	
5.1.9. Test Reusability			c3	
5.2. Test Sub-Processes and Activities	c12s9, c1s12			
5.2.1. Test Planning Process	c12s1, c12s8			
5.2.2. Test Design and Implementation	c12s1, c12s3			
5.2.3. Test Environment Set-up and Maintenance	c12s6	c8s1	c13s2	
5.2.4. Controlled Experiments and Test Execution	c12s7		c4s7, c5s6	
5.2.5. Test Incident Reporting	c13s4, c13s9, c13s11	c8s3	c7s8	
5.3. Staffing	c16			
6. Software Testing in the Development Processes and the Application Domains		c8, c15	c4s8, c7	
6.1. Testing Inside Software Development Processes		c8	c7	
6.1.1. Testing in Traditional Processes	c18		c7	
6.1.2. Testing in Line with Shift-Left Movement		c3, c8s2		
6.2. Testing in the Application Domains		c15	c4s8	
7. Testing of and Testing Through Emerging Technologies				
7.1. Testing of Emerging Technologies	c10s10	c17, c18		
7.2. Testing Through Emerging Technologies	c3s9			
8. Software Testing Tools	c12s11		c7	
8.1. Testing Tool Support and Selection	c12s11		c7	
8.2. Categories of Tools	c1, c3, c4, c7, c8, c9, c12			

REFERENCES

- [1*] S. Naik and P. Tripathy, *Software Testing and Quality Assurance: Theory and Practice*, 1st ed: Wiley, 2008.
- [2*] I. Sommerville, *Software Engineering*, 10th ed., Addison-Wesley, 2016.
- [3] E.W. Dijkstra, *Notes on Structured Programming*, Technological University, Eindhoven, 1970.
- [4] ISO/IEC/IEEE 29119 — System and software engineering — Software testing, ed. 2022.
- [5] “ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary,” 2nd ed. 2017.
- [6] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, Chapter Six — Mutation Testing Advances: An Analysis and Survey, *Advances in Computers*, 112, 2019: 275-378.

- [7] M. Utting, B. Legeard, F. Bouquet, E. Fourneret, F. Peureux, and A. Vernotte, Recent advances in model-based testing, *Advances in Computers*, 101, 2016, pp. 53-120.
- [8] IEEE Std 1012-2016, IEEE Standard for System, Software, and Hardware Verification, and Validation, ed. 2016.
- [9] ISO/IEC 25010:2011, Systems and software engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models, ed. 2011.
- [10] ISO/IEC/IEEE 32675:2022 Information technology — DevOps — Building reliable and secure systems including application build, package and deployment.
- [11] Software Engineering Competency Model (SWECOM), v1.0, 2014.
- [12] ISO/IEC 20246:2017, “Software and systems engineering — Work product reviews”, 2017.
- [13] V. Riccio, G. Jahangirova, A. Stocco, et al., Testing machine learning based systems: A systematic mapping, *Empirical Software Engineering*, 25, 2020, pp. 5193-5254.
- [14*] C.Y. Laporte, and A. April, *Software Quality Assurance*, IEEE Computer Society Press, 1st ed., 2018.
- [15] S. Demi, R. Colomo-Palacios, and M. Sánchez-Gordón, Software Engineering Applications Enabled by Blockchain Technology: A Systematic Mapping Study, *Applied Sciences*, 11(7), 2021, pp. 2960.
- [16] K. Mao, L. Capra, M. Harman, and Y. Jia. A survey of the use of crowd-sourcing in software engineering, *Journal of Systems and Software*, 126, 2017, pp. 57-84.
- [17] A. Bertolino, G.D. Angelis, M. Gallego, B. García, F. Gortázar, F. Lonetti, and E. Marchetti, A systematic review on cloud testing, *ACM Computing Surveys (CSUR)*, 52(5), 2019, pp. 1-42.
- [18] R. Achary and P. Raj, *Cloud Reliability Engineering: Technologies and Tools*, CRC Press, 2021.
- [19*] J. Nielsen, *Usability Engineering*, 1st ed., Boston: Morgan Kaufmann, 1993.